TurboFetch

Нативное Android-приложение для скачивания видео напрямую с CDN без прокси и рекламы

1. Проблема

Существующие приложения для скачивания видео работают по одной схеме: трафик идёт через серверы самого сервиса. Из этого следуют три недостатка, которые раздражают пользователя.

- **Искусственное замедление.** Файл качается не с той скорости, которую отдаёт CDN: скорость режут, чтобы держать пользователя на сервисе.
- **Реклама.** Баннеры и межстраничные переходы в приложениях, которые по определению должны быть утилитой.
- **Зависимость от чужой инфраструктуры.** Если сервис лежит, не работает и приложение, хотя сам файл лежит на публичном CDN и доступен без всяких посредников.

2. Идея решения

TurboFetch не является агрегатором: он не хранит видео и не раздаёт его. Приложение умеет ровно три вещи — разобрать ссылку площадки, достать прямую ссылку на файл и скачать его напрямую с CDN, с той же скоростью, с какой отдаёт сам сервер.

Ключевое архитектурное требование: **постоянно работающего сервера нет и не требуется**. Всё, что нужно для разбора и скачивания, умещается в приложение и выполняется на устройстве. Это же делает возможным установку APK в обход магазина приложений.

3. Требования

Требование	Как выполнено
Скачивание напрямую с CDN, без прокси	HTTP-запрос идёт к CDN площадки; промежуточных серверов нет
Отсутствие рекламы	В приложении нет ни рекламных SDK, ни аналитики сторонних сервисов
Работа без выделенного сервера	Разбор ссылок и скачивание выполняются на устройстве
Список задач с очередью	Задания хранятся в Room, порядок задаётся перетаскиванием
Настройка качества и имени файла	Пресет качества и шаблон имени с живым предпросмотром
Поддержка YouTube, TikTok, X	Собственные резолверы для каждой площадки

Скачивание аудио в mp3	ffmpeg, собранный под arm64-v8a через NDK
Устойчивость к обрыву связи	Докачка через Range и повтор с экспоненциальной задержкой

4. Архитектура

Приложение разделено на слои так, чтобы движок заменялся независимо от интерфейса. Единственный шов между ними — интерфейс `DownloadEngine`.

Слой	Ответственность	Ключевые типы
ui	Экраны Compose и ViewModel'ы	QueueScreen, ShrineScreen, FilesScreen
domain	Модель задачи и правила	DownloadTask, MediaFormat, AppSettings
data	Room, хранилище, резолверы площадок	TaskDao, FileDao, TikTokResolver
worker	Фоновая загрузка через WorkManager	DownloadWorker, DownloadScheduler

4.1. Поток данных

1. Ссылка попадает в очередь — из «Поделиться», из поля ввода или из клика в браузере.
2. Резолвер площадки возвращает список форматов: для каждого — прямая ссылка, битрейт и необходимые заголовки.
3. Пользователь выбирает качество; для DASH-видео подбирается отдельная аудиодорожка.
4. WorkManager запускает загрузку в foreground-сервисе: видео и звук качаются отдельно и склеиваются через MediaMuxer.
5. Готовый файл записывается в MediaStore, а задача переезжает во вкладку «Скачанное».

5. Разбор ссылок

Это самая хрупкая часть проекта: каждая площадка периодически меняет разметку, и это ломает парсеры. Таблица ниже — состояние на момент написания документа.

Площадка	Метод	Особенности
YouTube	NewPipeExtractor (JitPack)	До 2160p. Видео DASH и звук качаются отдельно
TikTok	Разбор JSON в HTML страницы	Две схемы: api-data и __UNIVERSAL_DATA_FOR_REHYDRATION__
X (Twitter)	Эндпоинт cdn.syndication.twimg.com	Токен в URL не валидируется; слой заменяемый

6. Технические решения

6.1. Склейка видео и звука

YouTube отдаёт видео и звук отдельными потоками. Склеиваются они штатными MediaExtractor и MediaMuxer — без нативного кода и без перекодирования: потоки копируются как есть.

6.2. Конвертация в mp3

ffmpeg собирается из исходников через NDK r27c под arm64-v8a с выравниванием страниц памяти 16 КБ (требование Android 15). Бинарник кладётся в `nativeLibraryDir` под именем `lib*.so` — только оттуда Android разрешает запускать исполняемые файлы (политика W^X). Сборка воспроизводима: `tools/build-ffmpeg.sh`.

6.3. Устойчивость к обрывам

Обрыв связи не считается ошибкой: воркер повторяет попытку с экспоненциальной задержкой (30 с), а частично скачанный файл докачивается запросом Range с нужного смещения. Ссылка TikTok или YouTube подписана и живёт недолго, поэтому перед повтором она разбирается заново.

6.4. Хранение

Очередь задач и история скачанных файлов — две отдельные таблицы Room. После скачивания задача уходит из очереди и остаётся в истории: очередь нужна только для того, что ещё предстоит скачать. Миграции базы написаны явно, поэтому обновление приложения не стирает данные.

7. Сборка

Требуется JDK 17+ и Android SDK (platform 37, build-tools 36). Сборка идёт через Gradle-обёртку:

```
./gradlew :app:assembleDebug
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

Подробная инструкция, включая сборку ffmpeg, — в README.md.

8. Текущее состояние

Часть	Состояние
YouTube: разбор и скачивание до 2160p	Работает, проверено на устройстве
TikTok: разбор и скачивание	Работает, проверено на устройстве
X: разбор и скачивание	Работает, проверено на устройстве
mp3 через ffmpeg	Работает, бинарник собран под arm64-v8a
Склейка видео и звука (MediaMuxer)	Работает
Докачка после обрыва	Работает
Очередь, порядок, лимит параллельных загрузок	Работает
История скачанных файлов	Работает

Покрытие юнит-тестами	32 теста, все зелёные

9. Ограничения и дальнейшая работа

- Слой разбора X недолговечен: эндпоинт не документирован и может измениться без предупреждения.
- TikTok отдаёт разметку в двух схемах и периодически их меняет — парсер придётся обновлять.
- mp3 доступен только на arm64-v8a: бинарник ffmpeg собран под одну архитектуру.
- Качество YouTube ограничено тем, что отдаёт площадка без аутентификационных токенов.
- Приложение не публикуется в магазинах, поэтому распространяется установкой APK в обход.

Документ подготовлен автоматически из исходного кода проекта. Актуальные версии: main (основная) и fun-branch (экспериментальная, со святыней Питера Тиля и темой «Warm Paper»).